

Integration testing

INTRODUCTION TO TESTING IN JAVA



Maria Milusheva
Senior Software Engineer

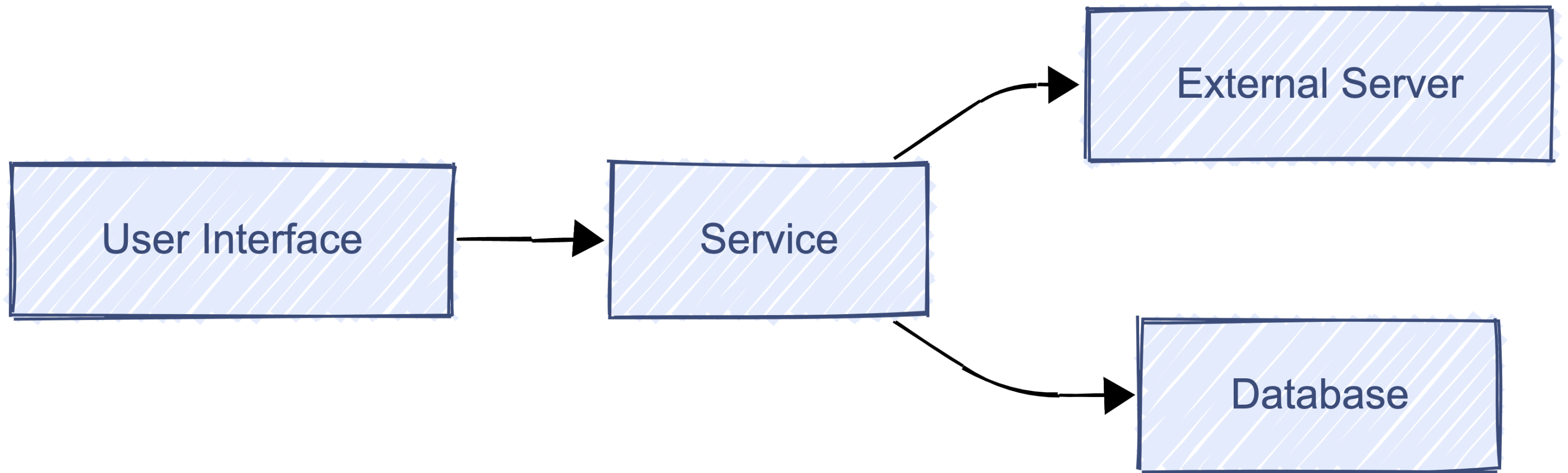
Defining integration testing

- **Integration** - the process of combining individual software components or subsystems into a unified system that functions cohesively.
- **Integration test** - a test that verifies the interaction between multiple components or modules of an application to ensure they work together correctly. It often involves testing with external dependencies.



Dependencies

- Software dependency - a relationship between software components where one component relies on the other to work properly



¹ Chart made using Mermaid: <https://mermaid.live>

Integration testing vs. unit testing

Unit tests:

- Test individual components in isolation
- Failures pinpoint issues with a specific part of the code
- Faster to write and run
- Use various tools to avoid testing dependencies
- Assert on exact results and outcomes

Integration tests:

- Test multiple components and their interactions
- Failures require deeper analysis to trace
- Can be heavy and take longer to set up and run
- Tests real interactions with real dependencies
- May assert on general behaviors instead of exact results

Example: Foreign exchange

Suppose we are building a currency exchange app:

```
public class ExchangeApp {  
    private EuropeanCentralBankServer bank; // Enables ExchangeApp to use the bank server  
  
    public ExchangeApp(EuropeanCentralBankServer bank) {  
        this.bank = bank; // Save the object passed through the constructor  
    }  
  
    public double convertEuroTo(String currency, double amount) {  
        double rate = this.bank.getRateEuroTo(currency);  
        return amount * rate; // Use the return values of the bank method in calculations  
    }  
}
```


Integration testing foreign exchange

Integration testing verifies both the `convertEuroTo` method and `EuropeanCentralBankServer` :

```
@Test
void convert_convertsWithoutError() {
    EuropeanCentralBankServer bank = new EuropeanCentralBankServer();
    ExchangeApp exchangeApp = new ExchangeApp(bank); // Pass bank object to constructor
    double amount = 1000.0;
    String currency = "USD";

    // convertEuroTo calls getRateEuroTo from the bank object
    double result = exchangeApp.convertEuroTo(currency, amount);

    assertTrue(result > 0); // Can't predict exact value, can only sanity test
}
```

Let's practice!

INTRODUCTION TO TESTING IN JAVA

Mocking and Stubbing in Unit Tests

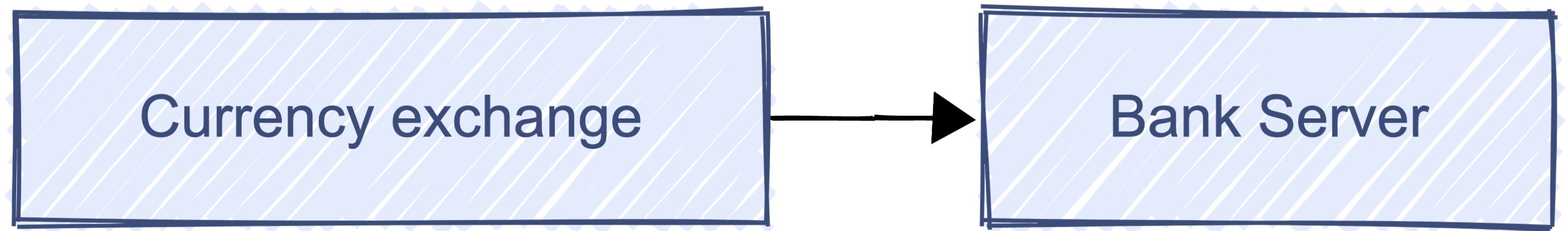
INTRODUCTION TO TESTING IN JAVA



Maria Milusheva
Senior Software Engineer

Motivation: Dependencies

Recall the dependency flow in the previous lesson: The currency exchange app depends on the bank server:



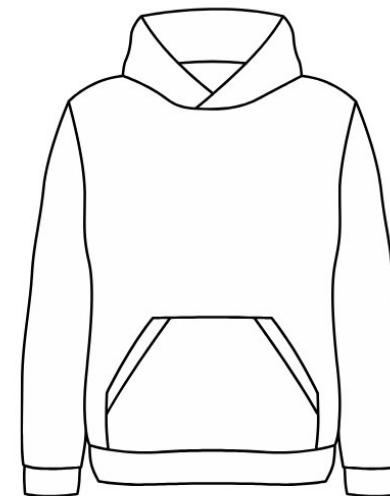
Impossible to verify calculations because **currency exchange rates are inherently unpredictable.**

Dependency management in testing

Mocking - creating simulated objects or behaviors for real objects that are difficult to use in tests, allowing us to test without relying on external dependencies.

Mockito - powerful Java library for creating mocks.

```
import static org.mockito.Mockito.mock;  
import static org.mockito.Mockito.when;
```



Mocking

- Mock = like a stand-in actor in a play, pretending to be the real object only as far as the test is concerned

Consider the previous lesson:

```
public double convertEuroTo(String currency, double amount) {  
    double rate = this.bank.getRateEuroTo(currency);  
    return amount * rate;  
}
```

→ Make predictable object for `EuropeanCentralBankServer` for testing

Mocking: Currency exchange

This is how we test with mocks:

```
@Test
void convert_returnsExpectedValue() {
    EuropeanCentralBankServer bank = mock(EuropeanCentralBankServer.class);
    ExchangeApp exchangeApp = new ExchangeApp(bank);
    // Need to program the mock so it works
    when(bank.getRateEuroTo("USD")).thenReturn(1.1);

    // Calls getRateEuroTo from the mock object, obtaining our pre-programmed values
    double result = exchangeApp.convertEuroTo("USD", 1000);

    assertEquals(1100, result); // We can now assert on exact return values
}
```

Common mistake

We should never forget to program our mocks:

```
@Test
void convert_returnsExpectedValue() {
    EuropeanCentralBankServer bank = mock(EuropeanCentralBankServer.class);
    ExchangeApp exchangeApp = new ExchangeApp(bank);

    // No mock programming - mock bank object doesn't know how to respond to getRateEuroTo
    double result = exchangeApp.convertEuroTo("USD", 1000);

    assertEquals(1100, result); // Will fail due to lack of mock setup
}
```

Stubbing

The test fails with:

```
org.opentest4j.AssertionFailedError:  
Expected :1100.0  
Actual   :0.0
```

Programming a mock is referred to as **stubbing**.

Stubbing - providing minimal, pre-defined answers to calls made during the test, usually not responding at all to anything outside what's programmed for the test.

Exceptions with Mockito

Mockito can also simulate error scenarios:

```
EuropeanCentralBankServer bank = mock(EuropeanCentralBankServer.class);
ExchangeApp exchangeApp = new ExchangeApp(bank);
// Instead of thenReturn() use thenThrow()
when(bank.getRateEuroTo("USD"))
    .thenThrow(new RuntimeException("Bank server unavailable."));

exchangeApp.convertEuroTo("USD", 1000); // getRateEuroTo will throw exception
```


Let's practice!

INTRODUCTION TO TESTING IN JAVA

Verification in mocking

INTRODUCTION TO TESTING IN JAVA



Maria Milusheva
Senior Software Engineer

Motivation: No return values

In the previous lesson and its exercises, we tested by asserting on return values.

- But what if there are no return values?
- What if the return values are not important or informative?

Example: saving to a database.



Example: Databases

Database - a digital repository for storing, managing and securing organized collections of data. There are many database providers:



- Database servers are durable, secure, reliable, and have unlimited capacity
- Real databases are too large and complex for unit tests (but may be used in integration tests)

Example: Log messages

Suppose we are processing logs:

```
public class MessageProcessor {  
    private InfoStore infoStore; // Will store info log messages here  
    private ErrorStore errorStore; // Will store error log messages here  
  
    public void saveMessage(String message) {  
        if (message.startsWith("[INFO]")) {  
            infoStore.save(message);  
        }  
        if (message.startsWith("[ERROR]")) {  
            errorStore.save(message);  
        }  
    }  
}
```

InfoStore and ErrorStore

Only need basic interfaces for the test to work:

```
// Blueprints for InfoStore and ErrorStore
// Mocks don't need the mocked classes to be properly implemented
interface InfoStore {
    void save(String message);
}

interface ErrorStore {
    void save(String message);
}
```

Mockito verify

How can we test a message was saved without creating any databases?

Assert a mock was used:

```
import static org.mockito.Mockito.verify;
```

Assert a mock was not used:

```
import static org.mockito.Mockito.verifyNoInteractions;
```


Test setup

```
@Test
void process_savesToInfoStore_whenInfoMessage() {
    InfoStore infoStore = mock(InfoStore.class);
    ErrorStore errorStore = mock(ErrorStore.class);
    MessageProcessor messageProcessor = new MessageProcessor(infoStore, errorStore);
    String message = "[INFO] Process started.";

    messageProcessor.saveMessage(message); // Will use either InfoStore or ErrorStore

    // Verify which one of the two databases was used
    verify(infoStore).save(message);
    verifyNoInteractions(errorStore);
}
```


Test failure messages

What if the message we create is instead:

```
String message = "[ERROR] Process failed!"
```

Then we will see test failures like this:

```
Wanted but not invoked:  
infoStore.save("[ERROR] Process failed!");  
Actually, there were zero interactions with this mock.
```

Further verification tricks

We can verify exactly how many times a log was invoked:

```
import static org.mockito.Mockito.times;
```

```
List<String> messages = new ArrayList<>(); // Create list and add elements
    messages.add("[INFO] Processing data...");
    messages.add("[INFO] Processing data...");
    messages.add("[INFO] Processing data...");

messageProcessor.saveMessageList(messages); // Save the three messages

verify(infoStore, times(3)).save("[INFO] Processing data...");
```

Let's practice!

INTRODUCTION TO TESTING IN JAVA